

National University of Computer and Emerging Sciences

Lab Manual

Data Analysis and Visualization



Lab 01

Instructor

Rida Mahmood, Asif Ahsan

Class

BDS-5A

Semester

Fall 2023

Fast School of Computing

FAST-NU, Lahore, Pakist

Objectives

After performing this lab, you will be able to understand:

- Python data Types, operators (all 3 types)
- Python loops (if-else, for, while)
- Python functions
- Python Arrays vs Python Lists
- Python Libraries used for data analysis (numpy, matplotlib, seaborn)

Numerical Python: **numpy**

Review the concepts on numpy: Scientific **Python** code uses a fast array structure, called the **numpy** array. Those who have worked in **Matlab** will find this very natural. For reference, the **numpy** documentation can be found here: [numpy](#).

Let's make a numpy array.

```
import numpy as np
```

```
my_array = np.array([1,4,9,16])
```

```
my_array
```

```
array([ 1,  4,  9, 16])
```

1. Numpy arrays support the same operations as lists! Below we compute length, slice, and iterate.
Run this code snippet in python

```
print("len(array):", len(my_array)) # Length of array
```

```
print("array[2:4]:", my_array[2:4]) # A slice of the array
```

```
# Iterate over the array
```

```
for ele in my_array:
```

```
    print("element:", ele)
```

```
### *** write the output of above code snippet***
```

In general you should manipulate numpy arrays by using numpy module functions (e.g. **np.mean**). This is for efficiency purposes, and a discussion follows below this section.

2. You can calculate the mean of the array elements either by calling the method **.mean** on a numpy array or by applying the function **np.mean** with the **numpy** array as an argument.

In [7]:

```
# Two ways of calculating the mean

print(my_array.mean())

print(np.mean(my_array))

#### *** write the output of above code snippet***
```

The way we constructed the `numpy` array above seems redundant. After all we already had a regular `python` list. Indeed, it is the other ways we have to construct `numpy` arrays that make them super useful.

3. There are many such `numpy` array *constructors*. Here are some commonly used constructors. Look them up in the documentation.

```
zeros = np.zeros(10) # generates 10 floating point zeros
```

```
zeros
```

```
#### *** write the output of above code snippet***
```

`Numpy` gains a lot of its efficiency from being strongly typed. That is, all elements in the array have the same type, such as integer or floating point. The default type, as can be seen above, is a float of size appropriate for the machine (64 bit on a 64 bit machine).

```
zeros.dtype
```

```
#### *** write the output of above code snippet***
```

```
np.ones(10, dtype='int') # generates 10 integer ones
```

```
#### *** write the output of above code snippet***
```

If the elements of an array are of a different type, `numpy` will force them into the same type (the longest in terms of bytes)

In [11]:

```
mixed = np.array([1, 2.3, 'eleni', True])
```

```
#elements are of different type
```

```
print(type(1), type(2.3), type('eleni'), type(True))
```

```
# all elements will become strings
```

```
mixed
```

```
#### *** write the output of above code snippet***
```

Often you will want random numbers. Use the `random` constructor!

```
np.random.rand(10) # uniform on [0,1]
```

```
### *** write the output of above code snippet***
```

4. You can generate random numbers from a normal distribution with mean 0 and variance 1 using `np.random.randn`:

```
normal_array = np.random.randn(10000)

print("The sample mean and standard deviation are {0:17.16f} and {1:17.16f},
respectively.".format(np.mean(normal_array), np.std(normal_array)))
```

The sample mean and standard deviation are 0.0063187264046235 and 0.9939554202791167, respectively.

5. numpy supports vector operations

What does this mean? It means that to add two arrays instead of looping over each element (e.g. via a list comprehension as in base Python) you get to simply put a plus sign between the two arrays.

```
ones_array = np.ones(5)

twos_array = 2*np.ones(5)

ones_array + twos_array
```

```
### *** write the output of above code snippet***
```

6. Note that this behavior is very different from `python` lists, which just get longer when you try to + them.

```
first_list = [1., 1., 1., 1., 1.]

second_list = [2., 2., 2., 2., 2.]

first_list + second_list # not what you want
```

```
### *** write the output of above code snippet***
```

7. On some computer chips numpy's addition actually happens in parallel, so speedups can be high. But even on regular chips, the advantage of greater readability is important.

`Numpy` supports a concept known as *broadcasting*, which dictates how arrays of different sizes are combined together. There are too many rules to list all of them here. Here are two important rules:

1. Multiplying an array by a number multiplies each element by the number
2. Adding a number adds the number to each element.

```

normal_5_7 = 5.0 + 7.0 * normal_array
np.mean(normal_5_7), np.std(normal_5_7)
### *** write the output of above code snippet***

```

2D arrays

1. We can create two-dimensional arrays without too much fuss.

create a 2d-array by handing a list of lists

```

my_array2d = np.array([
    [1, 2, 3, 4],
    [5, 6, 7, 8],
    [9, 10, 11, 12]
])

```

you can do the same without the pretty formatting (decide which style you like better)

```

my_array2d = np.array([ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12] ])

```

3 x 4 array of ones

```

ones_2d = np.ones([3, 4])
print(ones_2d, "\n")

```

3 x 4 array of ones with random noise

```

ones_noise = ones_2d + 0.01*np.random.randn(3, 4)
print(ones_noise, "\n")

```

3 x 3 identity matrix

```

my_identity = np.eye(3)
print(my_identity, "\n")

```

```

### *** write the output of above code snippet***

```

```

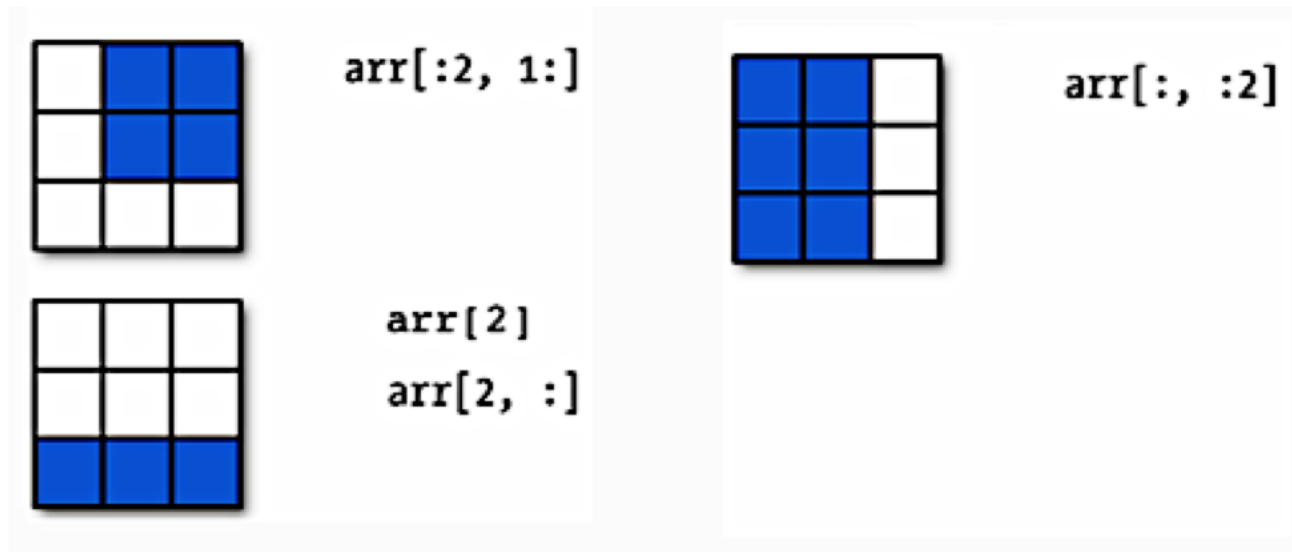
print(my_array2d)
print("element [2,3] is:", my_array2d[2, 3])

```

In [20]:

*** write the output of above code snippet***

2. Numpy arrays can be sliced, and can be iterated over with loops. Below is a schematic illustrating slicing two-dimensional arrays.



Notice that the list slicing syntax still works!

`array[2:, 3]` says "in the array, get rows 2 through the end, column 3]"

`array[3, :]` says "in the array, get row 3, all columns".

Numpy functions will by default work on the entire array:

`np.sum(ones_2d)`

*** write the output of above code snippet***

3. The axis `0` is the one going downwards (i.e. the rows), whereas axis `1` is the one going across (the columns). You will often use functions such as `mean` or `sum` along a particular axis. If you `sum` along axis `0` you are summing across the rows and will end up with one value per column. As a rule, any axis you list in the axis argument will disappear.

`np.sum(ones_2d, axis=0)`

*** write the output of above code snippet***

`np.sum(ones_2d, axis=1)`

*** write the output of above code snippet***

Exercise 1:

Create a two-dimensional array of size

3×5

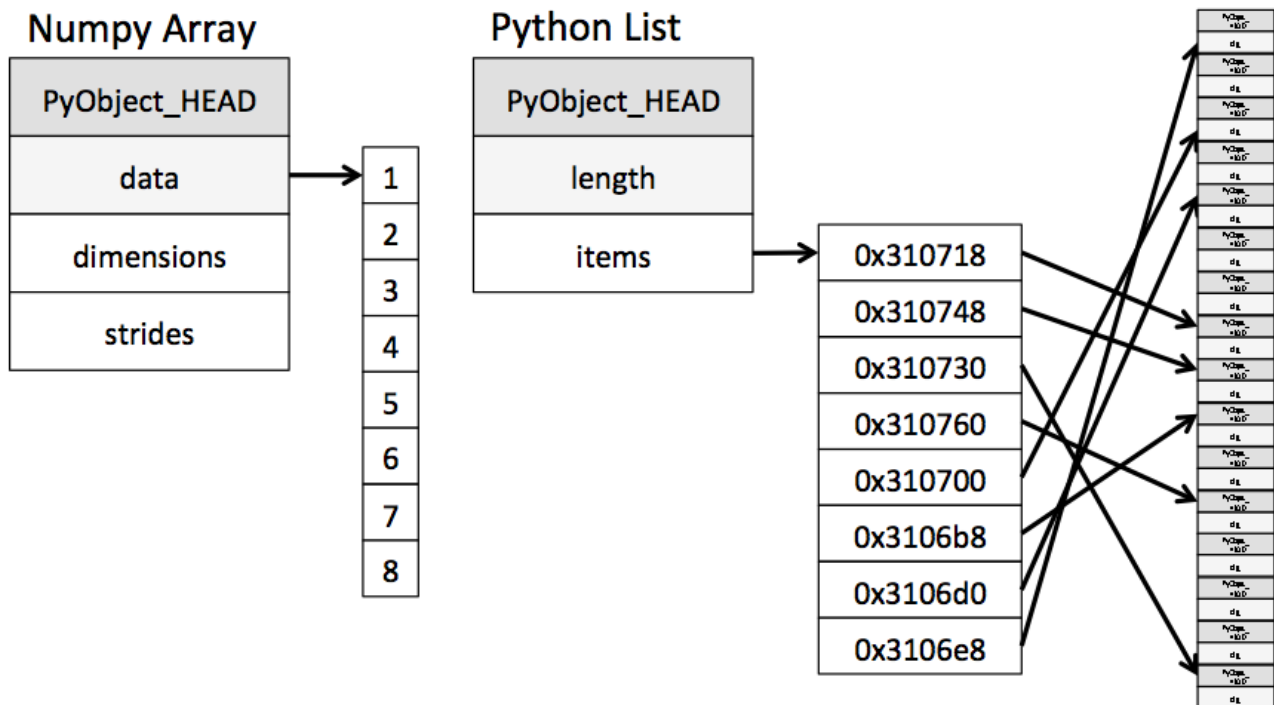
and do the following:

- Print out the array
- Print out the shape of the array
- Create two slices of the array:
 1. The first slice should be the last row and the third through last column
 2. The second slice should be rows
 3. 1–3
 4. and columns
 5. 3–5
 - 6.
- Square each element in the array and print the result

NumpyArrays vs. Python Lists?

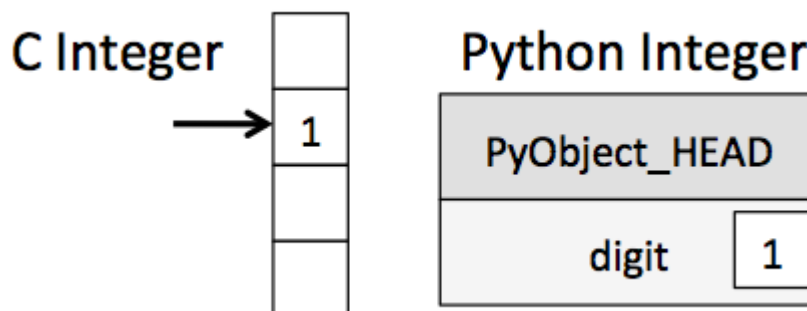
1. Why the need for **numpy** arrays? Can't we just use **Python** lists?
2. Iterating over **numpy** arrays is slow. Slicing is faster.

Python lists may contain items of different types. This flexibility comes at a price: **Python** lists store *pointers* to memory locations. On the other hand, **numpy** arrays are typed, where the default type is floating point. Because of this, the system knows how much memory to allocate, and if you ask for an array of size 100, it will allocate one hundred contiguous spots in memory, where the size of each spot is based on the type. This makes access extremely fast.

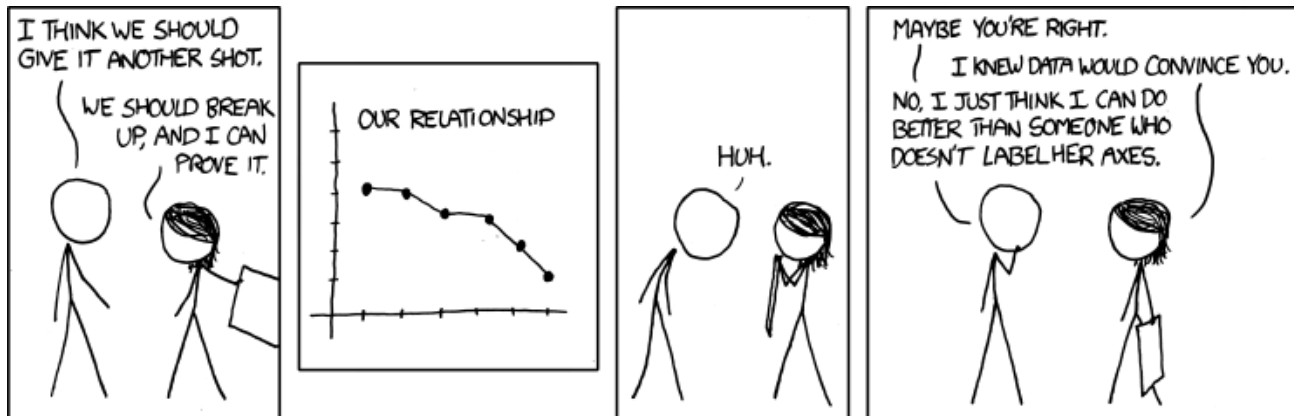


(image from Jake Vanderplas's Data Science Handbook)

Unfortunately, looping over an array slows things down. In general you should not access **numpy** array elements by iteration. This is because of type conversion. **Numpy** stores integers and floating points in **C-language** format. When you operate on array elements through iteration, **Python** needs to convert that element to a **Python int** or **float**, which is a more complex beast (a **struct** in **C** jargon). This has a cost.



2 - Plotting with matplotlib lib (and beyond)



Conveying your findings convincingly is an absolutely crucial part of any analysis. Therefore, you must be able to write well and make compelling visuals. Creating informative visuals is an involved process and we won't cover that in this lab. However, part of creating informative data visualizations means generating *readable* figures. If people can't read your figures or have a difficult time interpreting them, they won't understand the results of your work. Here are some non-negotiable commandments for any plot:

- Label x and y axes
- Axes labels should be informative
- Axes labels should be large enough to read
- Make tick labels large enough
- Include a legend if necessary
- Include a title if necessary
- Use appropriate line widths
- Use different line styles for different lines on the plot
- Use different markers for different lines

Before diving in, one more note should be made. We will not focus on the internal aspects of `matplotlib`. Today's lab will really only focus on the basics and developing good plotting practices. There are many excellent tutorials out there for `matplotlib`. For example,

- [matplotlib homepage](#)
- [matplotlib tutorial](#)

matplotlib

First, let's generate some data.

Exercise 2:

1. Use the following three functions to make some plots:

Logistic function:

$$f(z) = \frac{1}{1 + be^{-az}}$$

where a and b are parameters.

Hyperbolic tangent:

$$g(z) = b \tanh(az) + c$$

where a , b , and c are parameters.

Rectified Linear Unit:

$$h(z) = \begin{cases} z, & z > 0 \\ \epsilon z, & z \leq 0 \end{cases}$$

where $\epsilon < 0$ is a small, positive parameter.

You are given the code for the first function. Notice that z is passed in as a `numpy` array and that the functions are returned as `numpy` arrays. Parameters are passed in as floats.

```
def logistic(z: np.ndarray, a: float, b: float):
```

You should write a function to compute the rectified linear unit. The input should be a `numpy` array for z and a positive float for e .

2. However, the use of `ax.set_xlabel()` instead of `plt.xlabel()`. The difference is tiny, but you should be aware of it. I will use this plotting syntax from now on.

What else do we need to do to make this figure better? Here are some options:

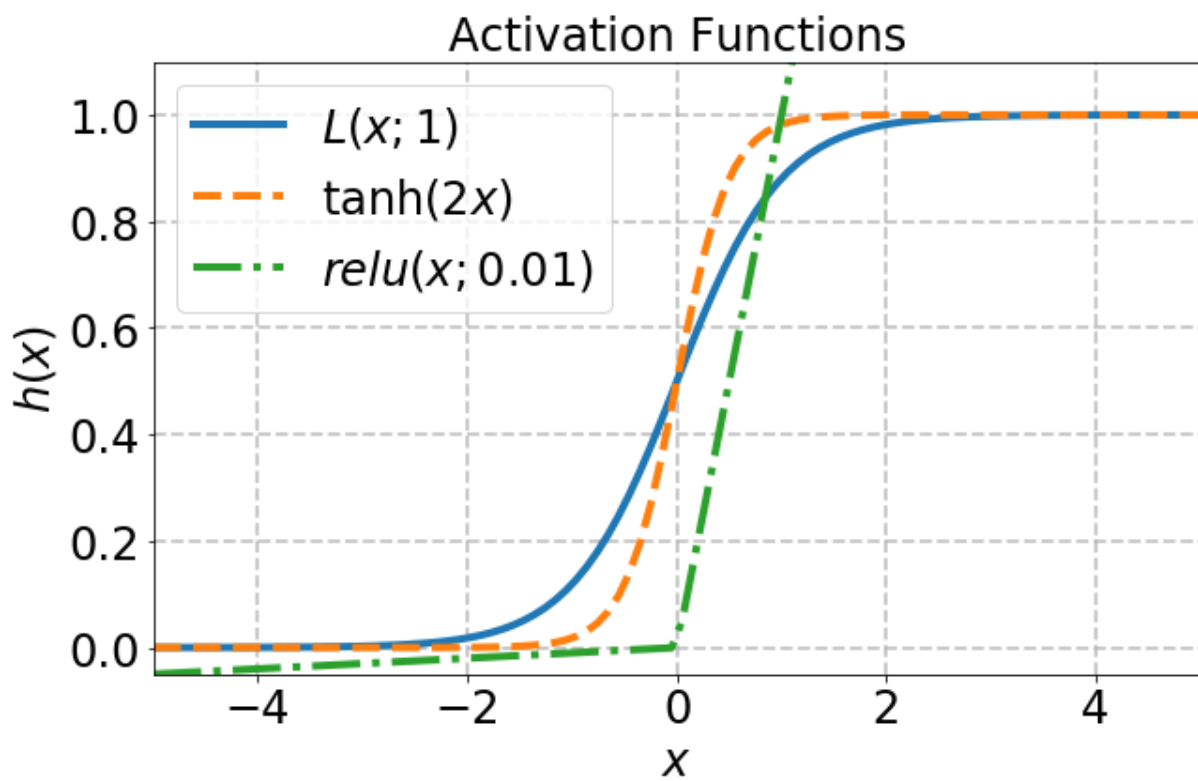
- Make labels bigger!
- Make line fatter
- Make the grid less pronounced
- Make a figure plot bigger

3. Do the following:

- Make a figure with the logistic function, hyperbolic tangent, and rectified linear unit.
- Use different line styles for each plot

- Put a legend on your figure

Figure should be displayed like this:



GOOD LUCK!!!!!!